Zadanie nr 1 - Generacja sygnału i szumu

Cyfrowe Przetwarzanie Sygnałów

Bartosz Kołaciński, 251554 Jakub Rosiak, 251620

29 marca 2026

1 Cel zadania

Celem zadania jest utworzenie programu umożliwiającego generację sygnałów i szumów o określonych parametrach, umożliwienie wykonywania na nich podstawowych operacji (dodawanie, odejmowanie, mnożenie i dzielenie) oraz ich graficzną prezentacje. Dodatkowo program umożliwia zapis i odczyt utworzonych sygnałów do plików binarnych.

2 Instrukcja korzystania z aplikacji

2.1 Instalacja aplikacji

Wymagania:

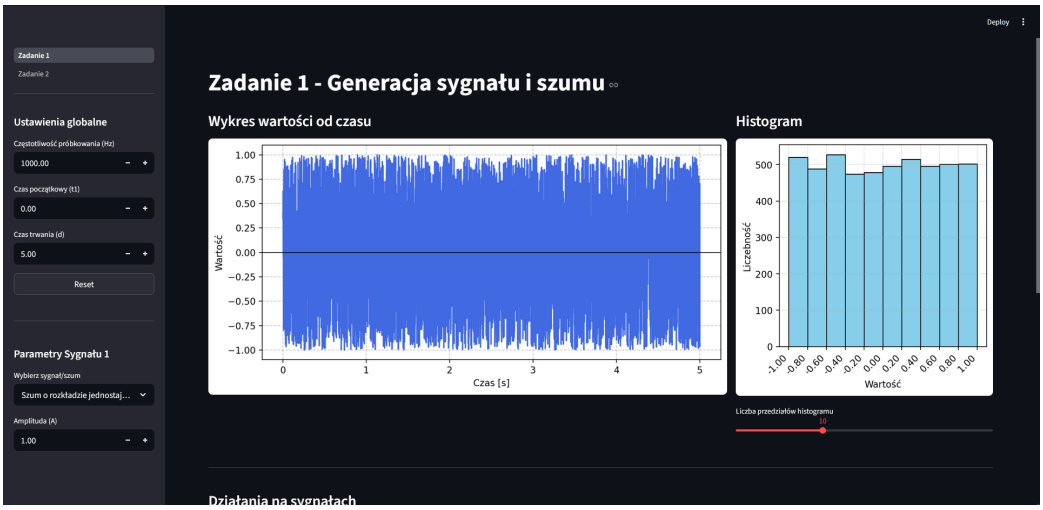
- git
- Python 3.13+

```
# Pobranie kodu i przejscie do folderu projektu
git clone https://github.com/bkolacinski/cyfrowe-przetwarzanie-
  sygnalow.git
cd cyfrowe-przetwarzanie-sygnalow

# Utworzenie wirtualnego srodowiska
python -m venv .venv

# Aktywacja wirtualnego srodowiska
.\.venv\Scripts\activate  # Windows
source .venv/Scripts/activate  # Linux / MacOS

# Pobranie zaleznosci
pip install -r requirements.txt

# Uruchomienie programu
streamlit run src/streamlit_app.py
```

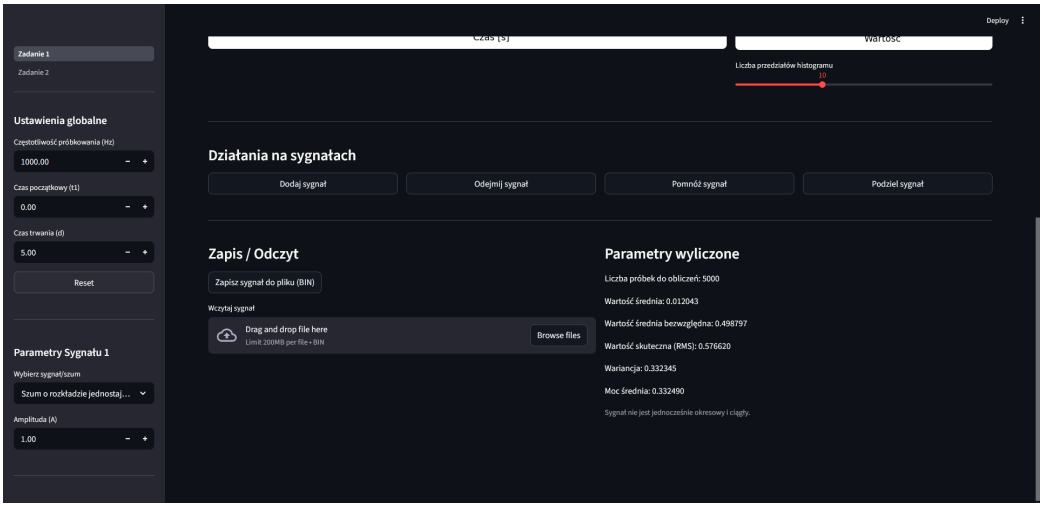
Po wykonaniu instalacji kodu oraz zależności, aby ponownie uruchomić program konieczne jest tylko wykonanie ostatniego polecenia (dodatkowo aktywacja wirtualnego środowiska, jeśli)

2.2 Korzystanie z aplikacji

Po wykonaniu ostatniego polecenia w przeglądarce powinno pojawić się nowa karta z aplikacją. Jeżeli tak się nie stanie należy wejść na `localhost:8501`.



Rysunek 1: Wygląd aplikacji w przeglądarce



Rysunek 2: Dalsza część aplikacji (po przewinięciu strony)

Panel po lewej stronie umożliwia:

- **Konfiguracja parametrów globalnych** – definiowanie ustawień wspólnych dla wszystkich przetwarzanych sygnałów.
- **Przywracanie ustawień** – przycisk „Reset” pozwala na szybki powrót do wartości domyślnych wszystkich parametrów.
- **Zarządzanie sygnałami** – wybór typu sygnału oraz szczegółowa konfiguracja jego parametrów.

Główna część strony umożliwia:

- **Wizualizację danych** – generowanie wykresu czasowego funkcji oraz odpowiadającego jej histogramu (z możliwością regulacji liczby przedziałów/słupków za pomocą suwaka).
- **Operacje na sygnałach** – dodawanie nowych składowych (domyślnie sygnał S1). W przypadku dużej liczby sygnałów panel boczny automatycznie udostępnia funkcję przewijania (scrollowania).
- **Archiwizację** – zapis bieżącej konfiguracji sygnału do pliku oraz odczyt wcześniej zapisanych danych.
- **Analizę parametrów** – bieżący podgląd wartości statystycznych i parametrów wyliczonych dla aktualnie wyświetlanego sygnału.

3 Metody generowania danych

Do generowania punktów wykorzystywana jest funkcja `get_time_vector()`, która tworzy równomiernie rozłożone próbki w przedziale czasu od chwili początkowej t_1 do chwili końcowej $t_1 + d$, gdzie d to czas trwania sygnału. Liczba próbek zależy od częstotliwości próbkowania f_s.

```
def get_time_vector(t1, d, fs):
        num_samples = int(d * fs)
        t = np.linspace(t1, t1 + d, num_samples, endpoint=True)
        return t
```

Funkcje [?]

4 Opis implementacji

Aplikacja została zaimplementowana w języku Python, z wykorzystaniem kilku kluczowych bibliotek wspierających zarówno logikę obliczeniową, jak i warstwę wizualną.

Do budowy interfejsu użytkownika wykorzystano bibliotekę Streamlit, która umożliwia szybkie tworzenie interaktywnych aplikacji webowych. Generowanie wykresów realizowane jest przy użyciu biblioteki Matplotlib, natomiast operacje numeryczne i przetwarzanie danych wykonywane są z pomocą NumPy. Do zapisu i odczytu danych z plików zastosowano bibliotekę msgpack, zapewniającą wydajną serializację danych.

W trakcie implementacji skupiono się na modularności kodu oraz czytelnym podziale odpowiedzialności pomiędzy poszczególne pliki.

Struktura projektu

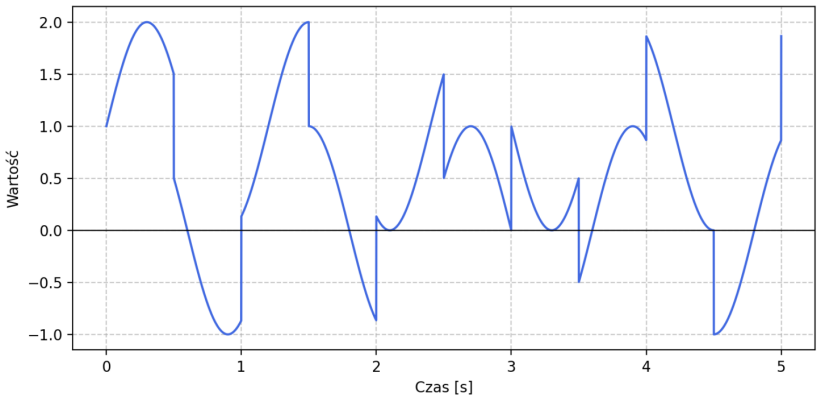
- `streamlit_app.py` – główny plik aplikacji, odpowiedzialny za konfigurację i uruchomienie interfejsu Streamlit
- `signal_io.py` – moduł obsługujący zapis i odczyt danych sygnałów z plików
- `functions.py` – zbiór funkcji generujących sygnały wykorzystywane w aplikacji
- `pages/zadanie1.py` – implementacja strony odpowiadającej zadaniu 1 w aplikacji

Funkcje wykorzystane w niniejszym sprawozdaniu zostały zaczerpnięte z instrukcji [**?**]. Niektóre wzory zostały uproszczone w celu czytelniejszej prezentacji, na przykład poprzez podstawienie wartości średniej i odchylenia standardowego w wzorze na szum gaussowski oraz jego skrócenie.

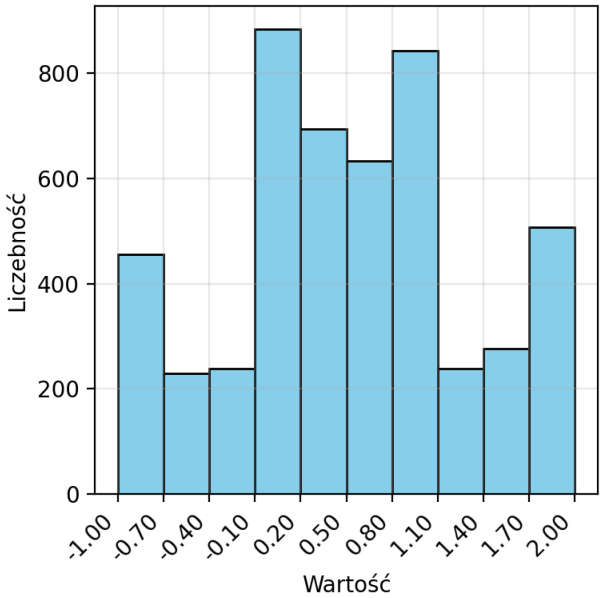
5 Eksperymenty

5.1 Eksperyment 1

Analizowano sygnał będący sumą sygnału sinusoidalnego o amplitudzie 1.0 i okresie 1.2 oraz sygnału prostokątnego o amplitudzie 1.0 i okresie 1.0. Celem było zaobserwowanie wpływu dodania sygnału prostokątnego na przebieg wynikowy.



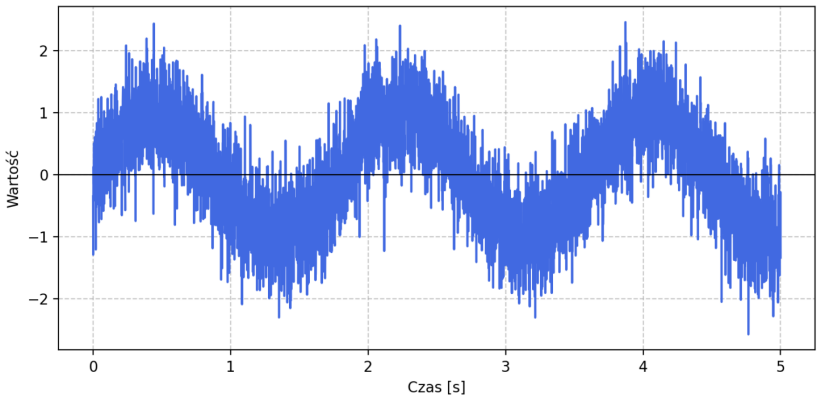
Rysunek 3: Przebieg sygnału wynikowego



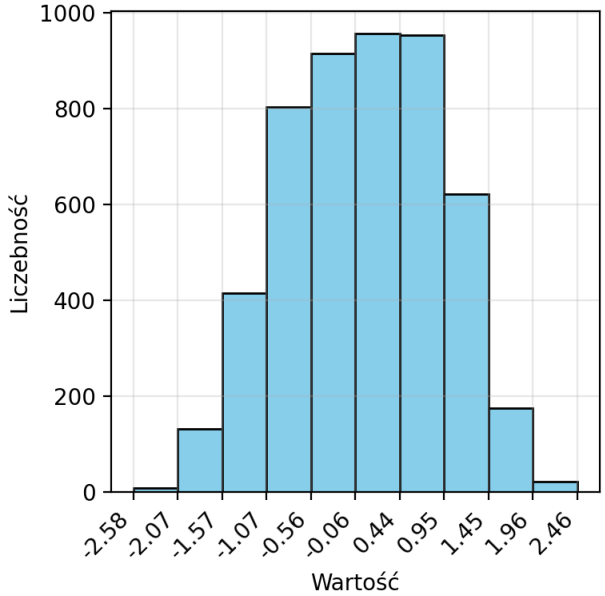
Rysunek 4: Histogram wartości sygnału

5.2 Eksperyment 2

Analizowano sygnał sinusoidalny o amplitudzie 1.0 i okresie 1.8, od którego odjęto szum gaussowski o amplitudzie 1.5. Celem było zbadanie wpływu szumu na kształt sygnału oraz jego rozkład wartości.



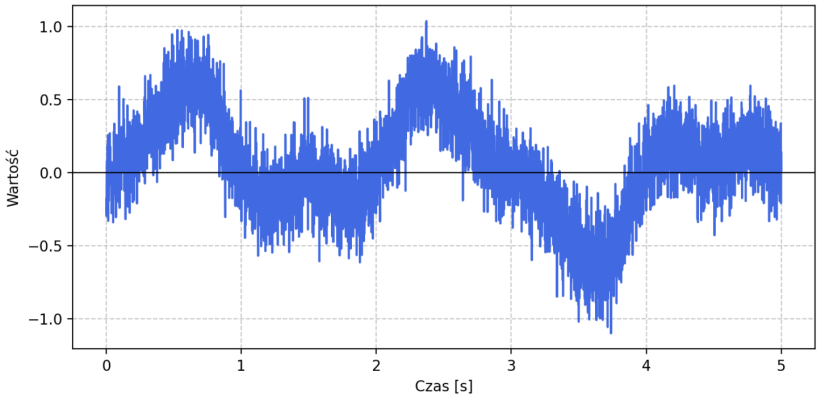
Rysunek 5: Przebieg sygnału z szumem



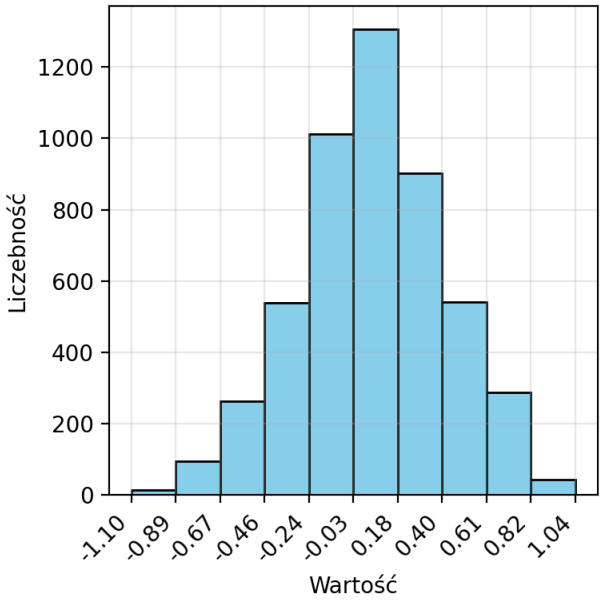
Rysunek 6: Histogram wartości sygnału

5.3 Eksperyment 3

Analizowano sygnał powstały jako iloczyn sygnału sinusoidalnego o amplitudzie 1.0 i okresie 2.0 oraz sygnału trójkątnego o amplitudzie 0.8 i okresie 1.5, z dodanym szumem gaussowskim o amplitudzie 0.5. Celem było zbadanie wpływu modulacji oraz szumu na przebieg i rozkład wartości sygnału.



Rysunek 7: Przebieg sygnału wynikowego



Rysunek 8: Histogram wartości sygnału

6 Wnioski

Zrealizowana aplikacja umożliwia intuicyjną generację oraz analizę sygnałów i szumów o zadanych parametrach. Zastosowanie interfejsu Streamlit znacząco upraszcza interakcję z użytkownikiem i pozwala na szybkie testowanie różnych konfiguracji.

Zaobserwowano, że zmiana parametrów takich jak częstotliwość próbkowania czy czas trwania sygnału ma bezpośredni wpływ na dokładność odwzorowania przebiegu. Z kolei dodanie szumu powoduje wyraźne zmiany w rozkładzie wartości, co jest widoczne na histogramach.

Implementacja podstawowych operacji arytmetycznych na sygnałach pozwala na łatwe badanie ich wzajemnych zależności. Dodatkowo możliwość zapisu i odczytu danych zwiększa funkcjonalność aplikacji i umożliwia powrót do wcześniej analizowanych przypadków.

Modularna struktura projektu ułatwia jego rozwój oraz utrzymanie.